Uni RV Universidade de Rio Verde	**AVALIAÇÃO 3 - GABARITO**	Nota

Curso *Engenharia de Software*	**Campus** *Rio Verde*
Disciplina *Banco de dados*	
Nome do(a) acadêmico(a)	**Valor** 3,0
Professor *Sergio Souza Novak*	**Turma** *B*
Data da Avaliação *23/06/2026*	**Código** *ESW417*

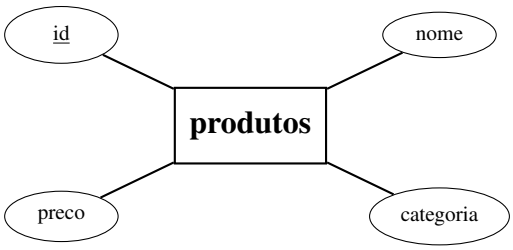
ATENÇÃO: Somente serão passíveis de **REVISÃO** avaliações resolvidas à **TINTA**.

ORIENTAÇÕES PARA A RESOLUÇÃO

- A avaliação é individual, sem consulta;
- A interpretação de texto faz parte da avaliação;

QUESTÕES

1. Em muitas aplicações web e mobile, a exibição de grandes volumes de dados é feita de forma dividida através de páginas (paginação) para melhorar o desempenho e a usabilidade. Considere um banco de dados de uma loja virtual com o seguinte modelo de entidade para os **produtos**:



   Escreva uma consulta SQL que retorne o `nome` e o `preco` dos produtos para exibir a **3ª página** de resultados, assumindo que cada página mostra exatamente **15 produtos**. Os resultados devem estar ordenados pelo `preco` de forma decrescente.

   Gabarito:
   ```
   SELECT nome, preco FROM produtos ORDER BY preco DESC LIMIT 15 OFFSET 30;
   ```
   (Também aceito o uso de `LIMIT 30, 15`)

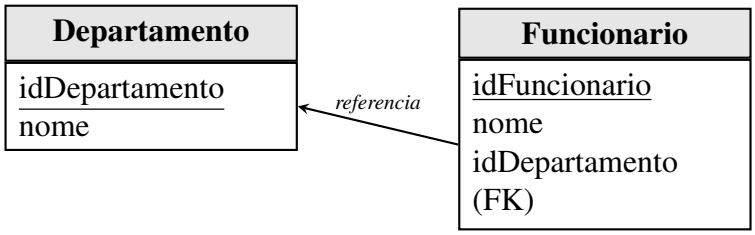
2. **Leia o texto de apoio abaixo para responder à questão:**

Ações Referenciais em SQL

Ações referenciais são regras aplicadas às restrições de chave estrangeira (FOREIGN KEY). Elas determinam como o banco de dados deve se comportar quando um registro "pai"(referenciado) é excluído ou atualizado. Servem para manter a integridade referencial de forma automática. As 5 ações padrão disponíveis no SQL são:

1. **CASCADE**: Propaga a alteração. Se o registro pai for deletado ou atualizado, todos os registros filhos correspondentes são automaticamente deletados ou atualizados.
2. **RESTRICT**: Bloqueia a operação. Impede que o registro pai seja deletado ou alterado caso exista algum registro filho que dependa dele.
3. **NO ACTION**: Semelhante ao RESTRICT na maioria dos SGBDs, rejeitando a alteração quando existem registros dependentes.
4. **SET NULL**: Define a chave estrangeira dos registros filhos como NULL. Para isso, a coluna não pode possuir a restrição NOT NULL.
5. **SET DEFAULT**: Define a chave estrangeira dos registros filhos para um valor padrão previamente estabelecido.

Considere o diagrama lógico e os dados das tabelas `Departamento` e `Funcionario` abaixo, em que cada funcionário pertence a um departamento por meio da chave estrangeira `idDepartamento`:



Departamento	
idDepartamento	nome
1	Financeiro
2	Recursos Humanos
3	TI

Funcionario		
idFuncionario	nome	idDepartamento
101	Ana	1
102	Bruno	3
103	Carla	3
104	Daniel	2

Suponha que o departamento de código `3` (TI) seja removido da tabela `Departamento`. Relacione cada ação referencial da coluna da esquerda com o comportamento correspondente na coluna da direita.

Ação Referencial	**Comportamento após remover o departamento 3**
(1) CASCADE	(**2**) A exclusão do departamento é impedida porque existem funcionários associados.
(2) RESTRICT	(**4**) Os funcionários Bruno e Carla permanecem cadastrados, mas seu campo `idDepartamento` passa a valer `NULL`.
(3) NO ACTION	(**1**) Os funcionários Bruno e Carla são removidos automaticamente.
(4) SET NULL	(**3**) A exclusão é rejeitada devido à existência de registros dependentes.
(5) SET DEFAULT	(**5**) O campo `idDepartamento` dos funcionários Bruno e Carla assume um valor padrão previamente definido.

Assinale a sequência correta de preenchimento dos parênteses.

a) 2 – 4 – 3 – 1 – 5

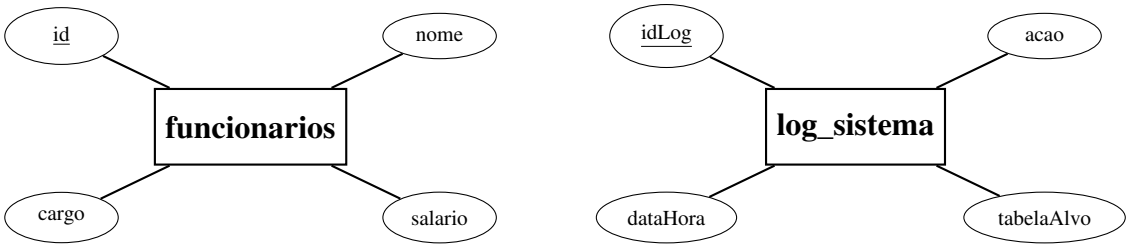
b) 1 – 4 – 2 – 3 – 5

c) 2 – 4 – 1 – 3 – 5

c) 2 – 5 – 1 – 3 – 4

d) 3 – 4 – 1 – 2 – 5

3. Considere o seguinte modelo de entidade das tabelas `funcionarios` e `log_sistema`:



Existe a necessidade de popular automaticamente a tabela de auditoria (`log_sistema`) toda vez que um registro for **inserido** ou **apagado** da tabela `funcionarios`. Assinale a alternativa que apresenta a configuração estrutural e sintática **CORRETA** de um **Trigger** (Gatilho) para essa finalidade:

a) O gatilho deve ser criado com `BEFORE SELECT ON funcionarios`, pois a leitura dos dados deve obrigatoriamente anteceder a gravação do log no sistema.

b) O gatilho deve utilizar a estrutura `AFTER INSERT OR DELETE ON funcionarios FOR EACH ROW`, executando internamente um `INSERT` na tabela de log para cada linha afetada.

b) A cláusula obrigatória para tabelas de auditoria é `FOR ALL TABLES`, garantindo que o gatilho seja acionado independentemente da tabela que sofreu a ação.

c) O código de um gatilho não pode executar comandos `INSERT INTO log_sistema`, pois gatilhos são objetos restritos exclusivamente à validação de dados em memória.

d) Para garantir que o log seja populado, o desenvolvedor deve disparar o gatilho manualmente através da aplicação usando o comando `EXECUTE TRIGGER trg_log`.

4. Os índices são estruturas fundamentais para otimizar o tempo de resposta das consultas. Sobre o índice **B-Tree** (Árvore B) e como verificar sua existência em um banco de dados, assinale a alternativa **CORRETA**:

a) O índice B-Tree organiza os dados de forma ordenada e hierárquica, acelerando muito buscas exatas e por intervalos (como `BETWEEN`). Podemos verificar os índices de uma tabela usando comandos específicos do SGBD (como `SHOW INDEX`).

a) O índice B-Tree organiza os dados de forma totalmente aleatória, tornando as consultas (`SELECT`) mais demoradas, mas acelerando a inserção (`INSERT`).

b) Índices B-Tree só podem ser criados em tabelas pequenas. Em tabelas muito grandes, com milhões de registros, o uso de índices é proibido e não é suportado pelo banco de dados.

c) É impossível listar ou verificar os índices de uma tabela após a sua criação, pois o SGBD esconde essas informações por questões de segurança.

d) A única maneira de saber se um índice existe é apagando a coluna da tabela e verificando se o banco de dados apresenta um erro de restrição.

5. Na otimização de bancos de dados, a **complexidade computacional** (expressa pela notação *Big-O*) é utilizada para descrever o desempenho de algoritmos de busca. Ao procurar um registro específico pelo valor de uma coluna, o SGBD pode realizar uma varredura sequencial completa (*Full Table Scan*) ou consultar um índice baseado em **Árvore B** (*B-Tree*). Assinale a alternativa que apresenta a complexidade de tempo de busca teórica correta para essas duas operações, respectivamente:

a) $O(1)$ para a varredura sequencial e $O(N)$ para a consulta no índice B-Tree.

b) $O(N)$ para a varredura sequencial e $O(\log N)$ para a consulta no índice B-Tree.

b) $O(\log N)$ para a varredura sequencial e $O(1)$ para a consulta no índice B-Tree.

c) $O(N^2)$ para a varredura sequencial e $O(N \log N)$ para a consulta no índice B-Tree.

d) $O(N)$ para ambas as estratégias, pois o índice afeta apenas a leitura física do disco, não alterando a complexidade do algoritmo de busca.